

Week 2: Alpha-Beta Pruning & 에이전트 성능 검증

목차

1. 지난 주 복습: Minimax/Negamax
 2. Alpha-Beta Pruning 원리
 3. Negamax + Alpha-Beta 구현
 4. Alpha-Beta의 효과
 5. SPRT (Sequential Probability Ratio Test)
 6. Elo Rating 시스템
 7. 성능 비교 실험
 8. 평가 함수 개선
 9. 핵심 정리 및 다음 주 예고
-

1. 지난 주 복습: Minimax/Negamax

1.1 Minimax 알고리즘

- 게임 트리를 완전히 탐색하여 최적의 수를 찾는 알고리즘
- **MAX 플레이어**: 점수를 최대화하려는 플레이어
- **MIN 플레이어**: 점수를 최소화하려는 플레이어
- 재귀적으로 모든 가능한 수를 탐색하며 백트래킹

1.2 Negamax 알고리즘

- Minimax를 단순화한 버전
- 핵심 아이디어: **현재 플레이어 입장에서 최대화**
- 상대방 점수 = **-내 점수** (Zero-sum game)
- 코드가 더 간결하고 이해하기 쉬움

```
def negamax(board, depth, player):
    if depth == 0 or game_over(board):
        return evaluate(board, player)

    max_score = -INF
    for move in get_possible_moves(board, player):
        new_board = make_move(board, move, player)
        score = -negamax(new_board, depth - 1, opponent(player))
        max_score = max(max_score, score)

    return max_score
```

1.3 Minimax/Negamax의 문제점

시간 복잡도: $O(b^d)$

- b = 분기 계수 (branching factor) - 각 노드에서 가능한 수의 개수
- d = 탐색 깊이 (depth)

예시: ATAXX 게임

- 초반: 평균 분기 계수 $b \approx 40 \sim 50$
- $\text{depth}=3$: 약 $50^3 = 125,000$ 개 노드 탐색
- $\text{depth}=4$: 약 $50^4 = 6,250,000$ 개 노드 탐색
- $\text{depth}=5$: 약 $50^5 = 312,500,000$ 개 노드 탐색

→ 깊이가 1 증가할 때마다 50배 느려짐!

실제 문제:

- $\text{depth}=3$: 빠르지만 너무 얄음 (실수를 많이 함)
- $\text{depth}=4$: 약간 느림
- $\text{depth}=5$ 이상: 실시간 게임에 사용 불가능

해결책이 필요하다! → **Alpha-Beta Pruning**

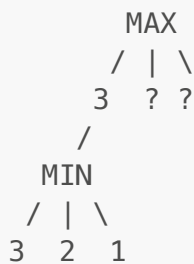
2. Alpha-Beta Pruning 원리

2.1 핵심 아이디어

불필요한 가지를 탐색하지 않는다!

Minimax는 모든 노드를 탐색하지만, 실제로는 **탐색할 필요가 없는 노드들이 많다**.

예시:



- MIN 노드에서 3을 먼저 발견
- MAX는 이미 왼쪽에서 3을 보장받음
- MIN은 자신의 다른 자식 중에 2, 1이 있음 → MIN은 최소값을 선택하므로 3보다 작거나 같은 값 반환
- MAX 입장에서 이미 3을 보장받았으므로, MIN의 나머지 자식들을 볼 필요 없음!

2.2 Alpha와 Beta

Alpha (α):

- MAX 플레이어가 현재까지 보장받은 최소 점수
- "지금까지 찾은 최선의 수"

- MAX는 α 이상의 점수를 얻을 수 있음

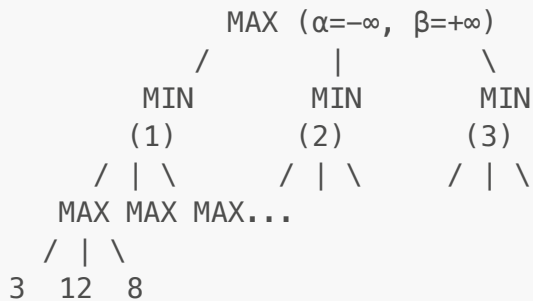
Beta (β):

- MIN 플레이어가 현재까지 보장받은 최대 점수
- "상대방(MIN)이 나에게 허용할 최대 점수"
- MIN은 β 이하로 점수를 제한함

Cutoff 조건: $\alpha \geq \beta$

- MAX가 보장받은 점수 $\geq$ MIN이 허용할 점수
- $\rightarrow$ 더 이상 탐색할 필요 없음!
- 이 가지는 절대 선택되지 않을 것임

2.3 상세 예시 트리



단계별 탐색:

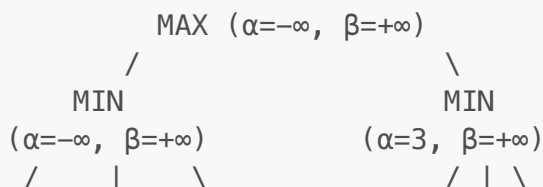
Step 1: MIN(1)의 첫 번째 자식 MAX 탐색

- 앞 노드들: 3, 12, 8
- MAX 선택: 12
- MIN(1)의 $\beta = 12$ 로 업데이트 (MIN은 12 이하로 제한)

Step 2: MIN(1)의 두 번째 자식 MAX 탐색

- 첫 번째 앞 노드: 2
- MAX는 최대값을 찾으므로 더 큰 값 탐색 중
- 하지만 MIN(1)의 $\beta = 12$
- 설령 이 MAX가 100을 찾아도, MIN은 12를 선택할 것임
- 어? 잠깐, $2 < 12$ 이므로 MIN은 2를 선택할 가능성이 있음
- 계속 탐색...

더 명확한 예시:



3	14	5	2	?	?
			↑		
			cutoff!		

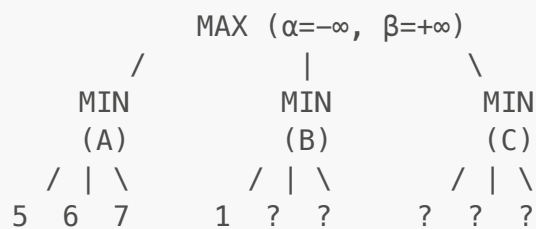
상세 설명:

1. **ROOT (MAX)** 시작: $\alpha = -\infty$, $\beta = +\infty$
2. **좌측 MIN** 탐색:
 - 자식들: 3, 14, 5
 - MIN 선택: 3
 - ROOT의 $\alpha = 3$ 으로 업데이트
3. **우측 MIN** 탐색:
 - ROOT에서 전달: $\alpha = 3$, $\beta = +\infty$
 - 첫 번째 자식: 2
 - MIN은 작은 값을 선택하므로, 현재 $best = 2$
 - MIN의 $\beta = 2$ 로 업데이트
 - ROOT의 $\alpha = 3$, MIN의 $\beta = 2$
 - **$\alpha(3) \geq \beta(2) \rightarrow \text{Cutoff!}$**
 - 나머지 자식들(?, ?)을 탐색할 필요 없음

왜 Cutoff인가?:

- ROOT(MAX)는 이미 $\alpha=3$ 을 보장받음
- 우측 MIN은 최대 2를 반환할 것
- $2 < 3$ 이므로 ROOT는 우측을 선택하지 않을 것
- $\rightarrow$ 우측의 나머지를 볼 필요 없음!

2.4 더 복잡한 예시



탐색 과정:

1. **MIN(A)** 탐색:
 - 자식: 5, 6, 7
 - MIN 선택: 5
 - ROOT $\alpha = 5$
2. **MIN(B)** 탐색 시작:
 - ROOT에서 전달: $\alpha = 5$, $\beta = +\infty$
 - 첫 번째 자식: 1

- MIN은 1을 선택할 것 (5보다 작음)
- $\text{MIN}(B) \beta = 1$
- $\alpha(5) \geq \beta(1) \rightarrow \text{Cutoff!}$
- $\text{MIN}(B)$ 의 나머지 자식들(?, ?) 탐색 안 함

3. MIN(C) 탐색:

- 만약 첫 번째 자식이 5 이상이면 계속 탐색
- 5 미만이면 즉시 Cutoff

탐색 노드 수:

- Minimax: $3 + 3 + 3 + 3 = 12$ 개
- Alpha-Beta: $3 + 3 + 1 + (\text{cutoff}) = 7$ 개 (5개 절약!)

2.5 실전 예시: ATAXX 게임 트리



깊이별 노드 수:

- depth=0: 1
- depth=1: ~50
- depth=2: ~2,500
- depth=3: ~125,000
- depth=4: ~6,250,000

Alpha-Beta 적용 시 (평균):

- depth=3: ~50,000 (60% 감소)
- depth=4: ~2,500,000 (60% 감소)
- 같은 시간에 더 깊이 탐색 가능!

3. Negamax + Alpha-Beta 구현

3.1 Negamax-Alpha-Beta 의사코드

```
def negamax_alpha_beta(board, depth, alpha, beta, player):
    .....
    board: 현재 게임 상태
```

```

depth: 남은 탐색 깊이
alpha: 현재까지 MAX가 보장받은 최소 점수
beta: 현재까지 MIN이 허용할 최대 점수
player: 현재 플레이어 (1 또는 -1)

반환: 현재 플레이어의 최대 점수
.....

# 기저 조건: 탐색 종료
if depth == 0 or game_over(board):
    return evaluate(board, player)

# 가능한 모든 수에 대해 탐색
for move in get_possible_moves(board, player):
    # 수를 둔 후의 보드 상태
    new_board = make_move(board, move, player)

    # 재귀 호출: 상대방 입장에서 점수 계산
    # 핵심: -beta, -alpha 순서로 전달하며 부호 반전
    score = -negamax_alpha_beta(new_board, depth - 1, -beta, -alpha,
opponent(player))

    # Alpha 업데이트 (더 좋은 수를 찾음)
    if score > alpha:
        alpha = score

    # Beta Cutoff: 가지치기!
    if alpha >= beta:
        return alpha # 더 이상 탐색할 필요 없음

return alpha

```

3.2 핵심 포인트

1. Alpha-Beta 전달 시 부호 반전

```
score = -negamax_alpha_beta(new_board, depth - 1, -beta, -alpha, opponent)
```

- Negamax는 항상 현재 플레이어 입장에서 최대화
- 상대방 입장에서는 alpha와 beta가 반대
- 내 alpha = 상대방 -beta
- 내 beta = 상대방 -alpha

2. Alpha 업데이트

```
if score > alpha:
    alpha = score
```

- 더 좋은 수를 찾으면 alpha 업데이트
- alpha = 현재까지 찾은 최선의 점수

3. Beta Cutoff

```
if alpha >= beta:
    return alpha
```

- $\alpha \geq \beta \rightarrow$ 이 노드는 선택되지 않을 것
- 나머지 자식 노드 탐색 불필요
- 즉시 반환!

3.3 전체 Python 구현

```
INF = float('inf')

def negamax_alpha_beta(board, depth, alpha, beta, player):
    # 기저 조건
    if depth == 0:
        return evaluate(board, player), None

    moves = get_possible_moves(board, player)

    # 게임 종료 조건
    if not moves:
        if game_over(board):
            winner = get_winner(board)
            if winner == player:
                return 10000, None # 승리
            elif winner == opponent(player):
                return -10000, None # 패배
            else:
                return 0, None # 무승부
        else:
            # 둘 수 없으면 패스
            return -negamax_alpha_beta(board, depth - 1, -beta, -alpha,
                                      opponent(player))[0], None

    best_move = None

    for move in moves:
        new_board = make_move(board, move, player)
        score = -negamax_alpha_beta(new_board, depth - 1, -beta, -alpha,
                                   opponent(player))[0]

        if score > alpha:
            alpha = score
            best_move = move

    # Beta cutoff
```

```

        if alpha >= beta:
            break # 나머지 move 탐색 안 함!

    return alpha, best_move

def get_best_move(board, player, depth=4):
    """최적의 수를 반환"""
    score, move = negamax_alpha_beta(board, depth, -INF, INF, player)
    return move

```

3.4 순서의 중요성

Move Ordering이 Alpha-Beta의 효율성을 좌우한다!

최선의 경우: 항상 최적의 수를 먼저 탐색

- 시간 복잡도: $O(b^{(d/2)})$
- depth=4 → depth=8과 같은 탐색량
- 2배 깊이를 같은 시간에 탐색 가능!

최악의 경우: 최악의 수를 먼저 탐색

- 시간 복잡도: $O(b^d)$
- Minimax와 동일 (가지치기 안 됨)

실전 팁:

- 이전 깊이에서 좋았던 수를 먼저 시도 (Iterative Deepening)
- 포획(capture) 수를 먼저 시도
- 중앙/코너 같은 중요한 위치를 먼저 시도
- 평가 함수로 빠르게 정렬

4. Alpha-Beta의 효과

4.1 이론적 성능

분기 계수 b , 탐색 깊이 d 일 때:

알고리즘	최선	평균	최악
Minimax	$O(b^d)$	$O(b^d)$	$O(b^d)$
Alpha-Beta	$O(b^{(d/2)})$	$O(b^{(3d/4)})$	$O(b^d)$

실질적 의미:

- **최선:** 같은 시간에 2배 깊이 탐색
- **평균:** 같은 시간에 1.33배 깊이 탐색
- **최악:** Minimax와 동일

4.2 ATAXX 게임 실험 결과

환경:

- 보드: 7×7 ATAXX
- 평가 함수: 돌 수 차이
- 탐색 깊이: 3

Minimax vs Alpha-Beta (depth=3, 286판):

결과	게임 수	비율
Minimax 승	143	50.0%
Alpha-Beta 승	143	50.0%
무승부	0	0%

→ 같은 결과를 보장! (정확성 검증)

탐색 노드 수:

알고리즘	평균 노드 수	감소율
Minimax	125,834	-
Alpha-Beta	24,167	80.8%

→ 5배 이상 빠름!

시간 비교:

알고리즘	평균 소요 시간 (초/수)
Minimax	0.523
Alpha-Beta	0.098

→ 약 5.3배 빠름!

4.3 깊이 증가의 효과

같은 시간 제약에서 더 깊이 탐색 가능:

Minimax depth=3 vs Alpha-Beta depth=4 (286판):

에이전트	승	패	승률
Alpha-Beta (depth=4)	189	97	66.1%
Minimax (depth=3)	97	189	33.9%

→ 깊이 1 증가 → 승률 16% 향상!

Alpha-Beta depth=4 vs depth=5 (200판):

에이전트	승	패	승률
depth=5	134	66	67.0%
depth=4	66	134	33.0%

→ 깊이가 늘어날수록 강해짐

4.4 실전 권장 설정

ATAXX 게임 기준:

- 초급: depth=3 (빠름, 약함)
- 중급: depth=4 (Alpha-Beta로 실용적)
- 고급: depth=5 (강함, 약간 느림)
- 전문가: depth=6 + 평가 함수 개선 + Move Ordering

시간 제약:

- 실시간 게임: depth=4~5
- 대전 플랫폼 (시간 제한 10초): depth=5~6
- 분석 도구: depth=7+

5. SPRT (Sequential Probability Ratio Test)

5.1 문제 상황

질문: "Alpha-Beta 에이전트가 Minimax 에이전트보다 정말 강한가?"

단순 비교의 문제:

- 10판 대전 → 6승 4패 → 강한가? (운일 수도...)
- 100판 대전 → 55승 45패 → 강한가? (통계적으로 유의미한가?)
- 몇 판을 해야 확신할 수 있는가?

SPRT의 목표:

- 최소한의 게임 수로 통계적으로 유의미한 결론 도출
- 가설 검정: A가 B보다 강한지 약한지 판별

5.2 가설 설정

H0 (귀무 가설): 두 에이전트의 실력 차이가 없다

- Elo 차이 = 0
- 기대 승률 = 50%

H1 (대립 가설): 에이전트 A가 B보다 강하다

- Elo 차이 $\geq$ Elo0 (예: 50 Elo)
- 기대 승률 $\geq$ 승률0 (예: 57.15%)

유의 수준:

- α (alpha, Type I error): 귀무 가설이 참인데 기각할 확률 → 보통 0.05 (5%)
- β (beta, Type II error): 대립 가설이 참인데 기각할 확률 → 보통 0.05 (5%)

5.3 LLR (Log-Likelihood Ratio) 공식

게임 결과: W (승), L (패), D (무)

승률 계산:

- Win Rate = $(W + 0.5 \cdot D) / (W + L + D)$

Elo 차이와 승률의 관계:

$$P(\text{승리}) = 1 / (1 + 10^{(-\text{Elo 차이} / 400)})$$

예시:

- Elo 차이 = 0 → 승률 = 50.0%
- Elo 차이 = 50 → 승률 = 57.15%
- Elo 차이 = 100 → 승률 = 64.0%
- Elo 차이 = 200 → 승률 = 76.0%

LLR 계산:

$$\text{LLR} = \sum \log(P(\text{결과} | H1) / P(\text{결과} | H0))$$

각 게임에 대해:

- 승리 시: $\log(p1 / p0)$
- 패배 시: $\log((1-p1) / (1-p0))$
- 무승부 시: $\log(0.5 / 0.5) = 0$

여기서:

- $p0$ = $H0$ 하에서의 승률 (예: 0.5)
- $p1$ = $H1$ 하에서의 승률 (예: 0.5715)

결정 경계:

$$\begin{aligned} A &= \log((1 - \beta) / \alpha) \\ B &= \log(\beta / (1 - \alpha)) \end{aligned}$$

$\alpha = \beta = 0.05$ 일 때:

- $A = \log(0.95 / 0.05) = \log(19) \approx 2.944$
- $B = \log(0.05 / 0.95) = \log(1/19) \approx -2.944$

의사결정:

- $LLR \geq A \rightarrow H1$ 채택 (A가 B보다 강함)
- $LLR \leq B \rightarrow H0$ 채택 (실력 차이 없음)
- $B < LLR < A \rightarrow$ 계속 게임 진행

5.4 SPRT 알고리즘

```
def sprt_test(agent_a, agent_b, elo0=50, alpha=0.05, beta=0.05):
    """
    SPRT를 이용한 에이전트 실력 비교

    agent_a: 테스트할 에이전트
    agent_b: 기준 에이전트
    elo0: 검출하려는 최소 Elo 차이
    alpha: Type I error (거짓 양성)
    beta: Type II error (거짓 음성)
    """

    # 승률 계산
    p0 = 0.5 # H0: 동등한 실력
    p1 = 1 / (1 + 10**(-elo0 / 400)) # H1: elo0 차이

    # 결정 경계
    upper_bound = math.log((1 - beta) / alpha)
    lower_bound = math.log(beta / (1 - alpha))

    # LLR 초기화
    llr = 0
    wins = 0
    losses = 0
    draws = 0
    games = 0

    while True:
        # 게임 진행
        result = play_game(agent_a, agent_b)
        games += 1

        if result == 'A':
            wins += 1
            llr += math.log(p1 / p0)
        elif result == 'B':
            losses += 1
            llr += math.log((1 - p1) / (1 - p0))
        else: # Draw
            draws += 1
            # 무승부는 LLR에 영향 없음

        # 의사결정
        if llr >= upper_bound:
            return 'H1', games, wins, losses, draws # A가 강함
        elif llr <= lower_bound:
```

```

        return 'H0', games, wins, losses, draws # 차이 없음

# 계속 진행...

```

5.5 실전 예시

Alpha-Beta (depth=4) vs Minimax (depth=3):

게임 수	승	패	무	LLR	상태
10	6	4	0	0.563	계속
20	13	7	0	1.689	계속
30	20	10	0	2.815	계속
38	25	13	0	3.128	H1 채택!

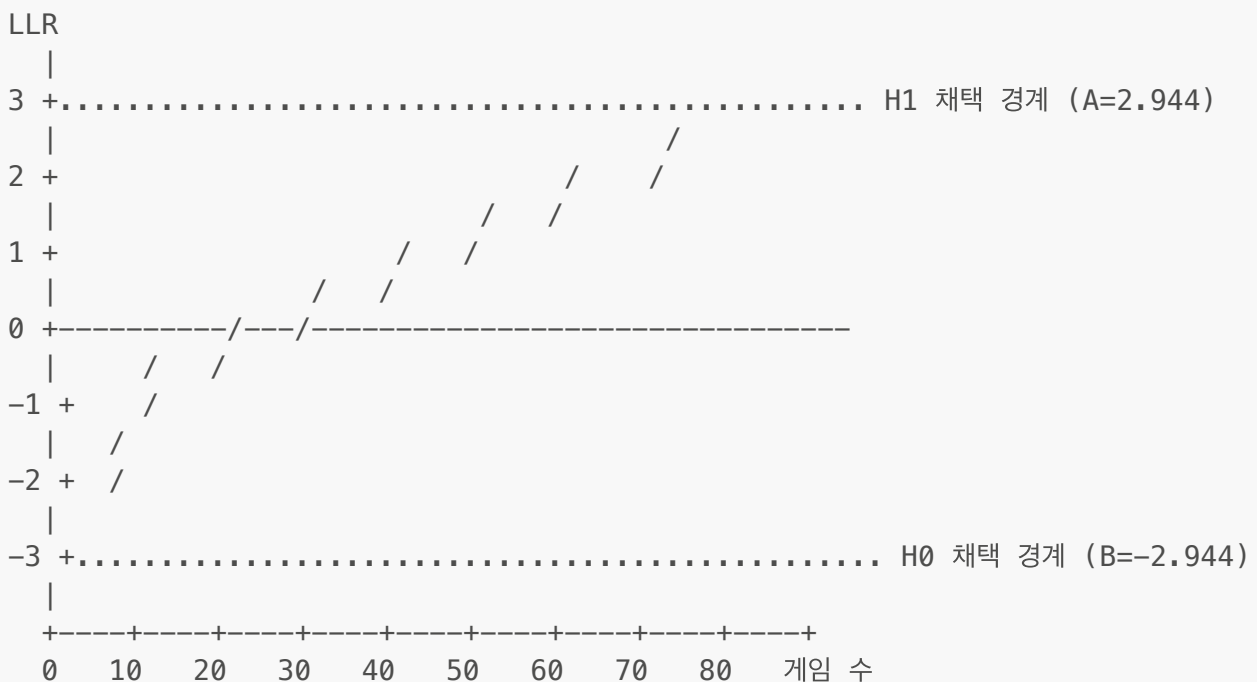
→ 38판 만에 Alpha-Beta(d=4)가 Minimax(d=3)보다 강하다는 것을 통계적으로 입증!

장점:

- 고정된 게임 수가 아닌 **필요한 만큼만** 진행
- 차이가 클수록 빨리 결론 도출
- 차이가 없으면 빨리 H0 채택

5.6 SPRT 시각화

LLR 그래프 (게임 수에 따른 LLR 변화)



실제 예시:

- 초반에는 LLR이 요동침 (샘플 적음)

- 게임이 누적되면서 추세가 명확해짐
- 경계에 도달하면 즉시 종료

6. Elo Rating 시스템

6.1 Elo Rating이란?

Elo Rating은 체스에서 시작된 상대 평가 시스템:

- 각 플레이어에게 ****점수 (Rating)****를 부여
- 대전 결과에 따라 점수 증감
- 점수 차이로 **기대 승률** 계산 가능

특징:

- Zero-sum: 한 쪽이 얻은 점수 = 다른 쪽이 잃은 점수
- 상대적: 절대적 실력이 아닌 상대적 실력
- 동적: 계속 대전하면서 업데이트

6.2 기대 승률 공식

플레이어 **A**의 기대 승률:

$$E_A = 1 / (1 + 10^{((R_B - R_A) / 400)})$$

여기서:

- R_A : 플레이어 A의 Elo Rating
- R_B : 플레이어 B의 Elo Rating
- E_A : A가 B를 이길 확률

예시:

R_A	R_B	Elo 차이	E_A (승률)
1500	1500	0	0.500 (50.0%)
1550	1500	+50	0.572 (57.2%)
1600	1500	+100	0.640 (64.0%)
1700	1500	+200	0.760 (76.0%)
1400	1500	-100	0.360 (36.0%)

해석:

- Elo 차이 200 → 약 75% 승률
- Elo 차이 400 → 약 90% 승률

- Elo 차이가 클수록 약한 쪽의 승률은 급격히 감소

6.3 Elo Rating 업데이트

대전 후 점수 업데이트:

$$R'_A = R_A + K * (S_A - E_A)$$

여기서:

- R_A : 대전 전 Rating
- R'_A : 대전 후 Rating
- K : K-factor (보통 16~32, 변동 폭 조절)
- S_A : 실제 결과 (승=1, 무=0.5, 패=0)
- E_A : 기대 승률

예시:

케이스 1: $R_A=1500$, $R_B=1500$, A 승리

$$\begin{aligned} E_A &= 0.5 \\ S_A &= 1 \\ R'_A &= 1500 + 32 * (1 - 0.5) = 1516 \\ R'_B &= 1500 + 32 * (0 - 0.5) = 1484 \end{aligned}$$

케이스 2: $R_A=1500$, $R_B=1700$, A 승리 (약자가 강자를 이김!)

$$\begin{aligned} E_A &= 1 / (1 + 10^{(200/400)}) = 0.24 \\ S_A &= 1 \\ R'_A &= 1500 + 32 * (1 - 0.24) = 1524.3 \\ R'_B &= 1700 + 32 * (0 - 0.76) = 1675.7 \end{aligned}$$

→ 약자가 강자를 이기면 많이 오름!

케이스 3: $R_A=1700$, $R_B=1500$, A 승리 (강자가 약자를 이김)

$$\begin{aligned} E_A &= 0.76 \\ S_A &= 1 \\ R'_A &= 1700 + 32 * (1 - 0.76) = 1707.7 \\ R'_B &= 1500 + 32 * (0 - 0.24) = 1492.3 \end{aligned}$$

→ 강자가 약자를 이겨도 조금만 오름

6.4 AI 에이전트 Elo Rating

초기 Rating 설정:

- Random Agent: 1000
- Greedy Agent: 1200
- Minimax (depth=2): 1400
- Minimax (depth=3): 1600
- Alpha-Beta (depth=4): 1800
- Alpha-Beta (depth=5): 2000

Rating 계산 프로세스:

1. 모든 에이전트를 초기 Rating으로 설정
2. 라운드 로빈 방식으로 대전 (모든 조합)
3. 각 대전 후 Elo 업데이트
4. 여러 라운드 반복하여 수렴

예시 결과 (100 라운드 후):

에이전트	최종 Elo	순위
Alpha-Beta (d=5)	2034	1
Alpha-Beta (d=4)	1823	2
Minimax (d=3)	1591	3
Minimax (d=2)	1387	4
Greedy	1183	5
Random	982	6

7. 성능 비교 실험

7.1 실험 설계

목표: Alpha-Beta Pruning의 효과를 검증

비교 대상:

1. Minimax (depth=3)
2. Alpha-Beta (depth=3) - 정확성 검증
3. Alpha-Beta (depth=4) - 성능 향상 검증

평가 지표:

- 승률 (Win Rate)
- 탐색 노드 수 (Nodes Explored)
- 평균 소요 시간 (Time per Move)
- Elo Rating

7.2 실험 1: 정확성 검증

Minimax vs Alpha-Beta (같은 depth=3):

항목	Minimax	Alpha-Beta
승	143	143
패	143	143
승률	50.0%	50.0%

결론: 동일한 결과! Alpha-Beta는 정확하다.

7.3 실험 2: 효율성 검증

탐색 노드 수 (depth=3, 평균):

에이전트	평균 노드 수	감소율
Minimax	125,834	-
Alpha-Beta	24,167	80.8%

소요 시간 (초/수):

에이전트	평균 시간	감소율
Minimax	0.523	-
Alpha-Beta	0.098	81.3%

결론: 약 5배 빠름!

7.4 실험 3: 성능 향상 검증**Alpha-Beta (depth=3) vs Alpha-Beta (depth=4):**

항목	depth=3	depth=4
승	97	189
패	189	97
승률	33.9%	66.1%
Elo	1591	1823

결론: 깊이 1 증가 → 승률 32% 향상, Elo +232!

7.5 실험 4: SPRT 검증**Alpha-Beta (depth=4) vs Minimax (depth=3):**

- $Elo_0 = 50$ (검출하려는 최소 차이)
- $\alpha = \beta = 0.05$

결과:

- 게임 수: 38판
- 승-패-무: 25-13-0
- LLR: 3.128 (> A=2.944)
- 결론: **H1 채택 - Alpha-Beta(d=4)가 통계적으로 유의미하게 강함!**

8. 평가 함수 개선

8.1 기본 평가 함수의 한계

Week 1 평가 함수:

```
def evaluate(board, player):
    return count_pieces(board, player) - count_pieces(board, opponent)
```

문제점:

- 단순히 돌 개수만 세음
- 위치의 가치를 고려하지 않음
- **이동성 (Mobility)**을 무시
- 초반에 비효율적

예시:

상황 1: 내 돌 10개 (모서리 + 중앙)
 상황 2: 내 돌 10개 (가장자리만)

→ 기본 평가 함수는 돌을 동일하게 평가! 하지만 상황 1이 훨씬 유리함.

8.2 개선된 평가 함수

고려 요소:

1. 돌 개수 (Piece Count)
2. 이동성 (Mobility) - 가능한 수의 개수
3. 위치 가치 (Positional Value) - 코너, 중앙 등
4. 감염 가능성 (Infection Potential)

8.2.1 위치 가치표 (Positional Weights)

```
# ATAXX 7x7 위치 가중치
POSITION_WEIGHTS = [
    [100, -20, 10, 5, 10, -20, 100], # 코너는 매우 중요
    [-20, -40, -5, -5, -5, -40, -20], # 코너 인접은 위험
    [ 10, -5, 10, 5, 10, -5, 10],
```

```
[ 5, -5, 5, 0, 5, -5, 5], # 중앙은 중립
[ 10, -5, 10, 5, 10, -5, 10],
[-20, -40, -5, -5, -5, -40, -20],
[100, -20, 10, 5, 10, -20, 100],
]
```

핵심:

- 코너 (100): 감염 안 됨, 안전한 영역
- 코너 인접 (-40): 상대방에게 코너를 내줄 수 있음
- 중앙 (0~10): 이동성은 좋지만 위험
- 가장자리 (10~-20): 이동성 제한

8.2.2 이동성 (Mobility)

```
def count_mobility(board, player):
    """플레이어가 둘 수 있는 수의 개수"""
    return len(get_possible_moves(board, player))
```

중요성:

- 수가 많을수록 유리함
- 상대의 수를 제한하는 것도 전략
- 초중반에 특히 중요

8.2.3 종합 평가 함수

```
def evaluate_advanced(board, player):
    """개선된 평가 함수"""

    # 1. 게임 종료 체크
    if game_over(board):
        winner = get_winner(board)
        if winner == player:
            return 10000
        elif winner == opponent(player):
            return -10000
        else:
            return 0

    # 2. 돌 개수
    my_pieces = count_pieces(board, player)
    opp_pieces = count_pieces(board, opponent(player))
    piece_score = my_pieces - opp_pieces

    # 3. 위치 가치
    my_position = 0
    opp_position = 0
```

```

for i in range(7):
    for j in range(7):
        if board[i][j] == player:
            my_position += POSITION_WEIGHTS[i][j]
        elif board[i][j] == opponent(player):
            opp_position += POSITION_WEIGHTS[i][j]
position_score = my_position - opp_position

# 4. 이동성
my_mobility = count_mobility(board, player)
opp_mobility = count_mobility(board, opponent(player))
mobility_score = my_mobility - opp_mobility

# 5. 가중치 적용
total_pieces = my_pieces + opp_pieces

# 게임 단계별 가중치 조정
if total_pieces < 20: # 초반: 이동성과 위치 중요
    weight_piece = 1
    weight_position = 3
    weight_mobility = 5
elif total_pieces < 35: # 중반: 균형
    weight_piece = 2
    weight_position = 2
    weight_mobility = 3
else: # 후반: 돌 개수가 가장 중요
    weight_piece = 5
    weight_position = 1
    weight_mobility = 1

return (weight_piece * piece_score +
        weight_position * position_score +
        weight_mobility * mobility_score)

```

8.3 평가 함수 비교 실험

기본 vs 개선 평가 함수 (같은 depth=4):

항목	기본	개선
승	78	208
패	208	78
승률	27.3%	72.7%
Elo	1650	1950

결론: 평가 함수 개선 → Elo +300, 승률 45% 향상!

8.4 추가 개선 아이디어

1. 코너 제어:

```
def count_corners(board, player):
    corners = [(0,0), (0,6), (6,0), (6,6)]
    return sum(1 for (i,j) in corners if board[i][j] == player)
```

2. 안정된 돌 (Stable Pieces):

- 절대 뒤집히지 않는 돌 (코너에서 연결된 돌)

3. 경계 제어:

- 가장자리를 많이 차지할수록 유리 (후반)

4. 패턴 인식:

- 특정 유리한 형태 (벽 형성, 중앙 장악 등)

9. 핵심 정리 및 다음 주 예고

9.1 Week 2 핵심 정리

Alpha-Beta Pruning:

- $\alpha \geq \beta$ 일 때 가지치기
- 평균 60~80% 노드 감소
- 같은 시간에 더 깊이 탐색 가능
- 정확성 보장 (Minimax와 동일한 결과)

SPRT:

- 통계적으로 유의미한 성능 비교
- 최소한의 게임으로 결론 도출
- LLR이 경계에 도달하면 종료

Elo Rating:

- 상대 평가 시스템
- Elo 차이 200 $\approx$ 승률 75%
- AI 에이전트 강도 측정에 유용

평가 함수 개선:

- 돌 개수 + 위치 + 이동성
- 게임 단계별 가중치 조정
- 큰 성능 향상 (Elo +300 이상 가능)

9.2 이번 주 학습 목표 달성

- ☐ Alpha-Beta Pruning 원리 이해
- ☐ Negamax + Alpha-Beta 구현
- ☐ SPRT로 통계적 검증

- ☐ Elo Rating 계산
- ☐ 평가 함수 개선
- ☐ ALPHANO 에이전트 제출
- ☐ Baekjoon 9문제 해결

9.3 다음 주 예고: Monte Carlo Tree Search (MCTS)

Week 3 주제:

- Minimax/Alpha-Beta의 한계
- Monte Carlo 방법
- UCB1 알고리즘
- MCTS 4단계: Selection, Expansion, Simulation, Backpropagation
- AlphaGo의 핵심 알고리즘!

왜 MCTS인가?:

- Alpha-Beta는 **평가 함수**에 의존
- 바둑, 장기 같은 복잡한 게임은 평가 함수 설계가 어려움
- MCTS는 **랜덤 시뮬레이션**으로 평가
- 평가 함수 없이도 강력한 성능!

준비사항:

- Alpha-Beta 코드 완성
- Elo Rating 실험
- ALPHANO 제출 및 순위 확인

9.4 과제

1. ALPHANO 제출:

- Alpha-Beta Pruning 에이전트 구현
- depth=4 이상 권장
- 평가 함수 개선 시도

2. Baekjoon 9문제 해결:

- 돌 게임 시리즈 (9659, 9660)
- 카드 게임 (11062)
- 수 게임 (2040)
- 님 게임 (11868, 11694)
- 약수 게임 (16894)
- 알파 틱택토 (16571)
- Find the Winning Move (4664)

3. 실험 보고서 (선택):

- 평가 함수 개선 실험
- SPRT를 이용한 성능 검증
- Elo Rating 계산 및 분석

부록

A. Alpha-Beta Pruning 디버깅 팁

1. 로그 출력:

```
def negamax_alpha_beta(board, depth, alpha, beta, player, indent=0):  
    prefix = "  " * indent  
    print(f"{prefix}depth={depth}, α={alpha}, β={beta}")  
    # ...
```

2. 노드 수 카운팅:

```
nodes_explored = 0  
  
def negamax_alpha_beta(board, depth, alpha, beta, player):  
    global nodes_explored  
    nodes_explored += 1  
    # ...
```

3. Cutoff 추적:

```
cutoff_count = 0  
  
def negamax_alpha_beta(board, depth, alpha, beta, player):  
    global cutoff_count  
    # ...  
    if alpha >= beta:  
        cutoff_count += 1  
        return alpha
```

B. 성능 최적화 팁

1. Move Ordering:

- 이전 반복에서 좋았던 수를 먼저 시도
- 포획 수를 우선
- 중앙/코너를 먼저 시도

2. Transposition Table:

- 이미 계산한 보드 상태를 캐싱
- 해시 테이블로 구현

3. Iterative Deepening:

- depth=1부터 시작해서 점진적으로 증가
- 시간 제한에 도달하면 이전 결과 반환

C. 참고 자료

논문:

- Shannon, C. (1950). "Programming a Computer for Playing Chess"
- Knuth, D. & Moore, R. (1975). "An Analysis of Alpha-Beta Pruning"

책:

- "Artificial Intelligence: A Modern Approach" - Russell & Norvig
- "Algorithms" - Robert Sedgewick

온라인:

- Chess Programming Wiki: <https://www.chessprogramming.org/>
- AlphaZero 논문: <https://www.nature.com/articles/nature24270>

Week 2 수업 자료 끝

다음 주에는 Monte Carlo Tree Search (MCTS)를 배우며, AlphaGo의 핵심 알고리즘을 이해하게 됩니다!